VISUALIZATION

* Data is just a numbers until we turn into a picture.

→ **Data visualization** :- It is the graphical representation of data to help people understand patterns, trends, relationships & insights quickly

With code

(1) Matplotlib, seaborn, Plotly

(2) Very high flexibity (can customize every element)

(3) can integrate into apps, ML pipelines.

With Tools

(1) Power BI, Tableau, Google data studio

(2) Limited to what the tool allow.

(3) Very tough to integrate with ML Pipelines.

1. MATPLOTLIB

→ **Intro** : creator = John Hunter (in 2003)

It is created to bring MATLAB-like plotting capabilities into python.

It serves as the core foundation for data visualization, allowing us to turn raw numbers into understandable pictures.

MATLAB is a high-level programming & numerical computing software used for Mathematics, data visualization, analysis, simulation & engineering applications.

Importing of Matplotlib

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

→ these are import for Basic structure.

→ for using arrays

→ for using dataFrames

→ **Basic plotting** : Most basic plot is a line graph, created using plt.plot().

In [1]:

```
x = np.linspace (0,5, 11)
x
y = x** 3
y =
```

output =

Out [2]:

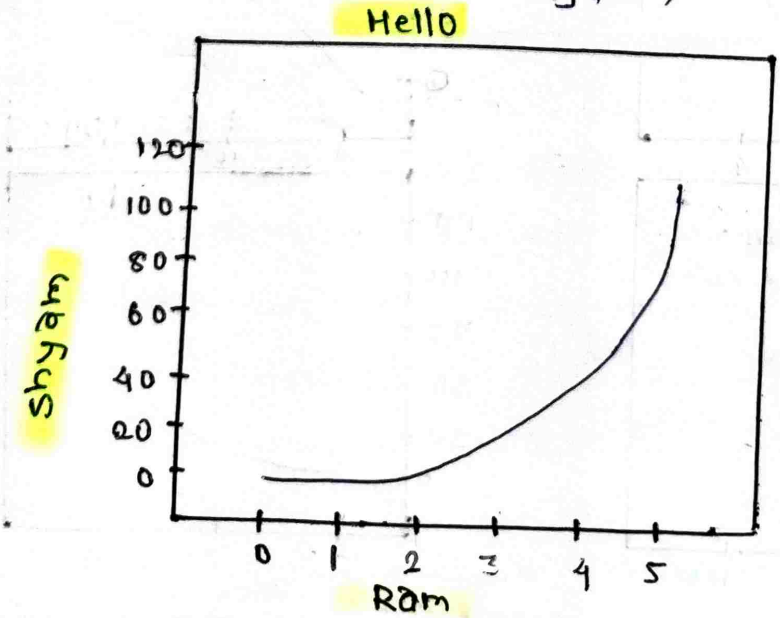
```
array( [ 0.0, 0.125 , 1.0 , 3.375 , 8.0 , 15.625 , 27.0,
47.875, 64.0, 91.125, 125.0 ])
```

In [2]

ex code =

```
plt.plot (x, y)  # for plot graph is x & y axis
plt.title ("Hello") # for naming title
plt.xlabel ("Ram") # for naming x-Axis
plt.ylabel ("shyam") # for naming y-Axis
```

output = Text (0, 0.5, "shyam")



→ **Subplots** : It allows to create multiple smaller plots side by side on a single blank canvas.

syntax:- `plt.subplot(rows, column, index_number)`

ex code =

In [3]:

```
plt.subplot(2, 2, 1)        # 2 = rows, 2 = column, 1 = index/graph no.
plt.plot(x,y)

plt.subplot(2,2,4)
plt.plot(x,y)

plt.subpot(2,2,3)
plt.plot(y,x)

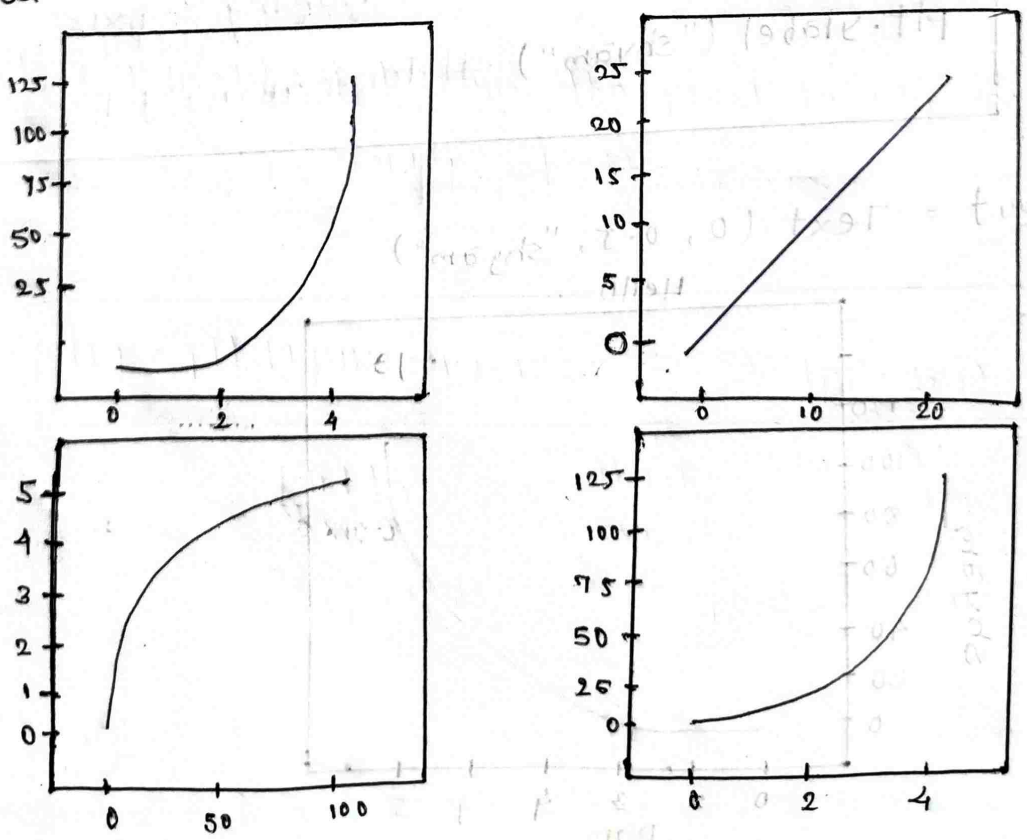
plt.subplot(2,2,2)
plt.plot(x**2, x**2)
```

index / graph no.

1	2
3	4

out[85]: [<matplotlib.lines.Line2D at 0x322d105f0>]

Output =



→ The object-oriented Way (Advanced control)

Instead of letting Matplotlib handle the canvas automatically, the object-oriented approach gives us full control over the figure (the blank canvas) & the axes (the actual plots). This allows for

Syntax:

```
axis1 = fig.add_axes([left, bottom, width, height])
```

- **Percentage Based sizing**: The four values must be decimals between 0 and 1, representing % of the canvas. For example, a width of "0.8" means the graph takes up 80% of canvas width.
- **Overlapping**: Because we dictate the exact coordinates, we can easily create overlapping plots, or picture-in-picture graphs, by placing a smaller axis inside the bounds of a larger one.

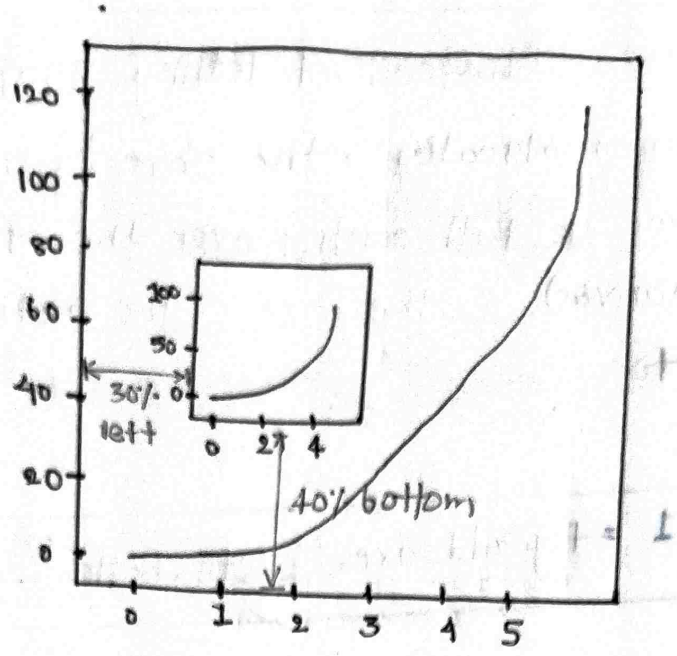
In [4]:

ex code =

```
fig = plt.figure()
axis1 = fig.add_axes([0.1, 0.1, 1, 1])
axis 1.plot(x,y)
axis 2 = fig.add_axes([0.3, 0.4, 0.3, 0.3])
axis 2.plot(x,y)
```

left, Bottom, width, height

Output =



→ **Fig size & Resolution (DPI)** : While creating a figure, we manually control its dimension & pixel clarity using `figsize` & `dpi` arguments.

- **Figsize** : Takes a tuple defining the width & height of the figure in inches.

- **dpi** : (Dots per inch) ; Determines the pixel density or clarity of the image.

ex code =

```
fig = plt.figure(figsize=(20,8), dpi=100)
```

Width: 20 X 100 = 2000 pixels
Height: 8 X 100 = 800 pixels
dpi

* As default dpi is always 100 more the no go forward more the image quality increases.

→ Types of plots

(1) Scatter plot (plt.scatter): Plots indivisual data points (x,y) without connecting them.

used to build & visualize linear regression algorithm in machine learning.

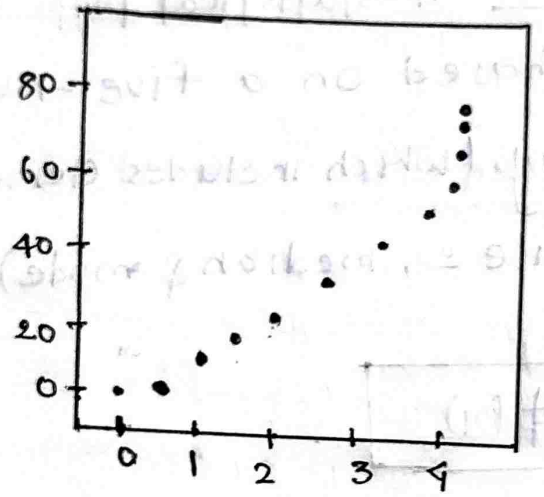
ex code =

In[5]:

```
plt.scatter(x,y)
```

output =

Out[5]:



(2) Histogram (plt.hist): Used to check the distribution of Quantative data.

In[6]:

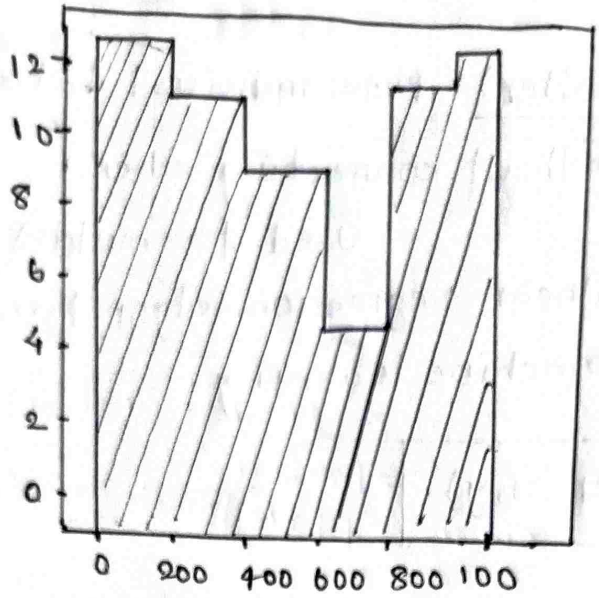
ex code =

```
from random import sample
data = sample(range(1,100),101)
plt.hist(data)
```

we can divide the data points into segments called "bins". here only only axis can represent

Output:

Out [6]:



* only for representational

(3) Box plot (plt.boxplot): A statistical plot that visualizes data based on a five-number based summary (which includes elements like Quartile 1, Quartile 2, median & mode)

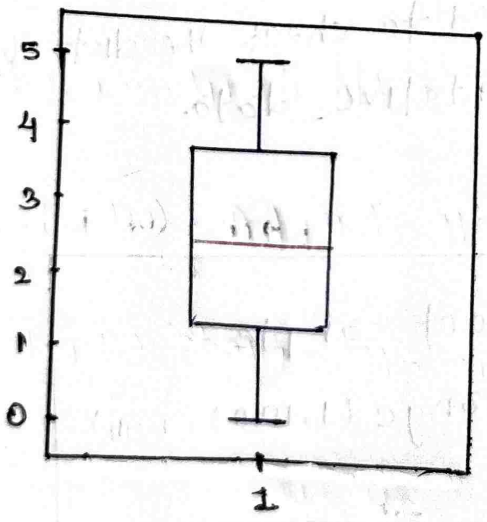
ex code =

In [7]:

```
plt.boxplot(x)
```

output:

Out [7]



→ Saving plots :- `plt.savefig("basicplot.png")`

after visulization is complete, we can easily save the figure to we local PC or machine an an image file.

ex code =

In [8]

```
fig = plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax.plot(x, y)
plt.savefig("basicplot.png")
```

*output file or png saved as "basicplot.png".

→ Working with Image: Matplot.lib has unique capability to load, manipulate & display images directy.

but we need to load image first like ("images.JPeg")

ex code =

In [9]

```
import matplotlib.image as mpimg        ↗ for import Image Module
img = mpimg.imread('images.JPeg')
plt.imshow(img)
plt.axis('off')
```

for load the Image

for displaying the Image

for hiding axis because Matplotlib automatically add axes x & y

• **cropping image** : for cropping image

Syntax:

```
cropped-img = img [row-start : row-end ,
                   column-start : column-end]
```

ex code =

```
In [10]:
cropped-img = img [50:200 , 100:300]
plt.imshow(cropped-img)
plt.axis ('off')
```

rowstart, columnstart, rowend, column end

→ **Advanced styling Parameter (optional)**

Customizing lines & markers is straightforward but relies on specific keyword arguments. we can alter the design by passing these directly into our "plt.plot" function.

• **color:** changes line/marker color.

• **linewidth (or lw)** : Adjusts the thickness of the line.

• **linestyle (or ls)**: changes the line pattern.

• **marker**: Add shapes to the exact data points.
eg- circles, squares, plus sign etc.

- markersize: Adjust how large the marker shape are.

- markerface color & markeredge color: Allows us to customize the inner colour & outer border color of the marker shapes.

2. DISTRIBUTION PLOTS

→ **Core concept of Distribution plots**

- **Purpose:** Distribution plots are specifically designed to visualize the distribution of (numerical & continuous) data.

*** Quantitative**

- Quantitative variables consist of continuous numerical values.

ex- age, bill like 200rs etc.

*** Qualitative**

- Qualitative (categorical) variables represent distinct categories.

ex- gender, days of the week, or smoker status. etc.

- **setup :-**

In [1]

```
import Matplotlib.pyplot as plt
import seaborn as sns
```

→ to generate plots

→ to loading datasets that are built-in seaborn like "tips".

→ **Histogram (sns.histplot or sns.distplot):** Best for showing how a single continuous variable is distributed. It works by dividing our data range into vertical column called "bins".

ex code =

In[2]

```
df = sns.load_dataset('tips')
df
```

→ used to load build-in dataset in seaborn like 'tips'.

In[3]

```
df['size'].unique()
```

output = array([2, 3, 4, 1, 6, 5])

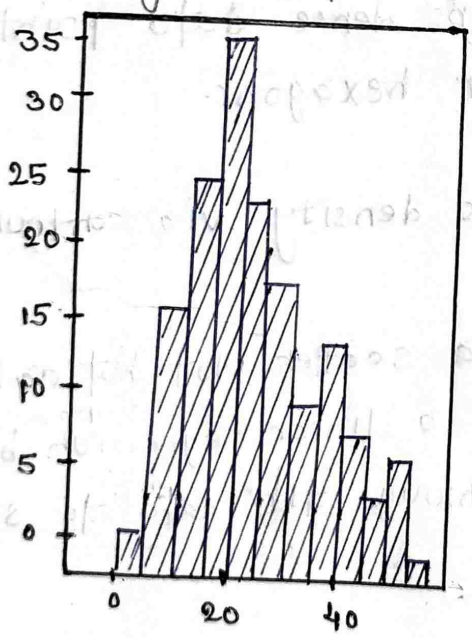
In[4]

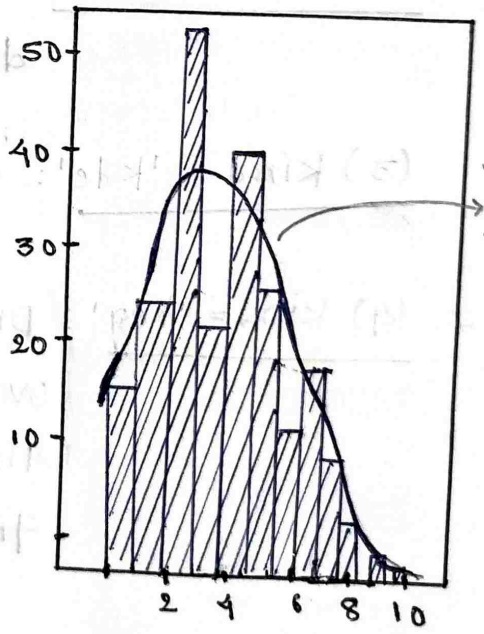
```
plt.subplot(1, 2, 1)
sns.histplot(df['total_bill'], bins=20, kde=False)
plt.subplot(1, 2, 2)
sns.histplot(df['tip'], bins=20, kde=True)
```

→ without this it always false

output = <Axes: xlabel='tip', ylabel='count'>

*only for representation, Not 100% accurate





bell curve line ∵ Kde=True

- **KDE (Kernal density Estimate)**:- we can add a `Kde=True` argument to draw smooth "bell curve" over the histogram

Use:- helps easily indentifies the skewness or normal distribution of our data.

→ Joint Plots (sns.jointplot)

If we wont to compare the distribution of two quantitative variables simultaneously.

It displays a main chart in the center to compare the variables, bordered by individual histograms on the X & y margin.

- **The "kind" parameter**: This alter the visual style of central comparision chart.

(1) **kind = 'scatter'**: The default style
Plotting individual dots for each intersecting data point.

(2) **kind = 'hex'**: Groups dense data points into darker hexagons.

(3) **kind = 'kde'**: shows density via contour liner.

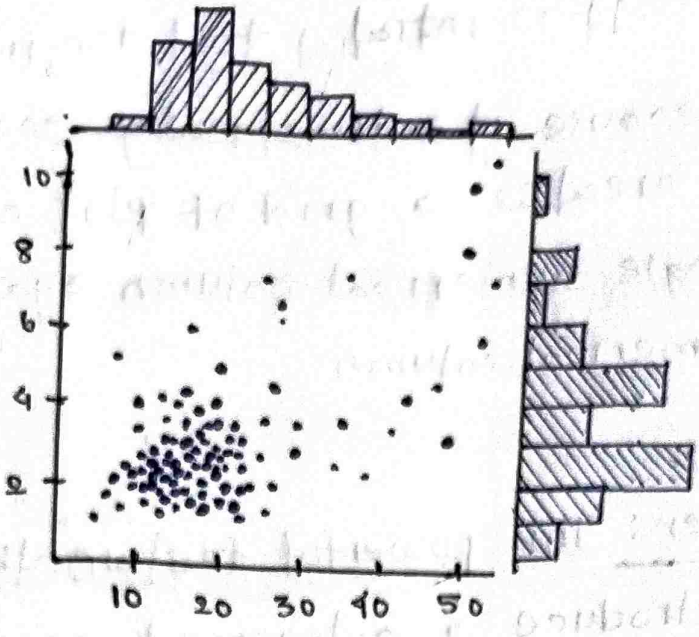
(4) **kind = 'reg'**: Draw a scatter plot but calculate & overlays a linear regression best fit line through the data to show the trend.

In[5]

ex code =

```
sns.jointplot(x='total_bill', y='tip', data=df ,
              kind='scatter')
```

Output = <seaborn.axisgrid.JointGrid at 0x3297e44d0>



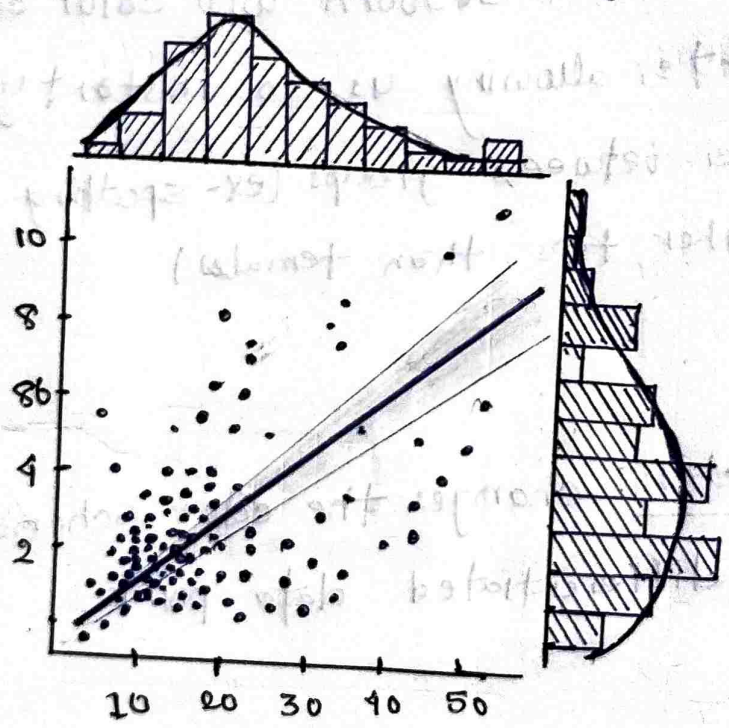
→ only for representation
Not 100% Accurate

In [6]:

```
sns.jointplot(x='total_bill', y='tip', data = df, kind='reg')
```

Output:

<seaborn.axisgrid.JointGrid at 0x323737e2e60>



- We Must specify which column goes to the X-axis & which goes to the Y-axis, along with dataset

→ Pair Plots (sns.pairplot)

It is essential tool for intial data analysis because it automatically scans entire dataset & creates a grid of plot comparing every single numerical column against every other numerical column

- **The "hue" parameter**: This powerful feature that allow us to introduce a categorical variable into our numerical plot.

By assigning hue to a category (like 'gender' or 'size').

Seaborn will color code the data points, allowing us to instantly spot differences between groups (ex- spotting if males left higher tips than females)

- **The 'palette' parameter**: changes the color scheme of the differentiated data points.

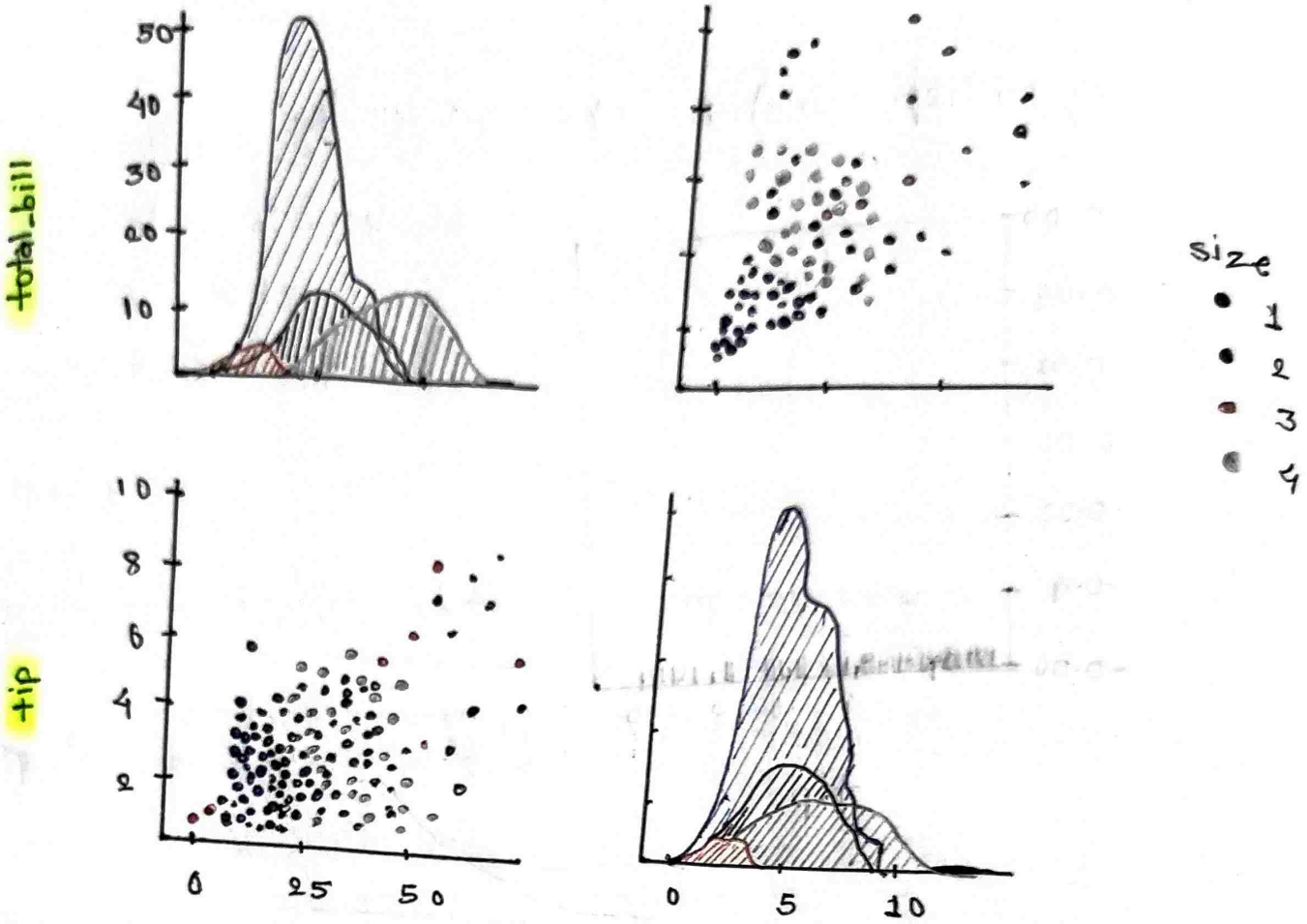
ex code =

In [1]:

```
sns.pairplot(df, hue='size', palette='rainbow')
```

Output = <seaborn.axisgrid(df, hue='size', palette='rainbow')

*only for representation, Not 100% Accurate



→ Rug plots (sns.rugplot):

It is a highly simplistic plot that does not use traditional x axis bins.

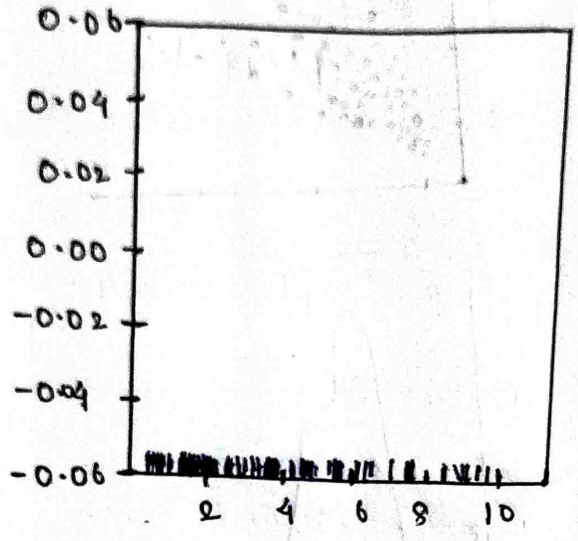
Instead, it draws a single vertical line (or tick mark) on the axis for every single data point that exists, making the axis look like the threads of a rug.

ex code = In [8]:

```
sns.rugplot(df['tip'])
```

output : <Axes : xlabel = 'tip' >

* only for representation, Not 100% Accurate.



3. CATEGORIAL PLOTS

→ Core concepts

It is used to visualize data where at least one variable is qualitative (categorial). These plots typically map a categorical variable on one axis against a quantitative (numerical) variable on the other to help us to distribution or aggregate data

• setup

In [1]:

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

- `plt` → for adding titles, labels, figure size & displays the plot.
- `sns` → create categorial plots
- `np` → used in case we need calculations or create sample data

In [2]:

```
df = sns.load_dataset("tips")
df
```

→ for load build-in data set (like-'tips') from seaborn

→ Count Plot (sns.countplot)

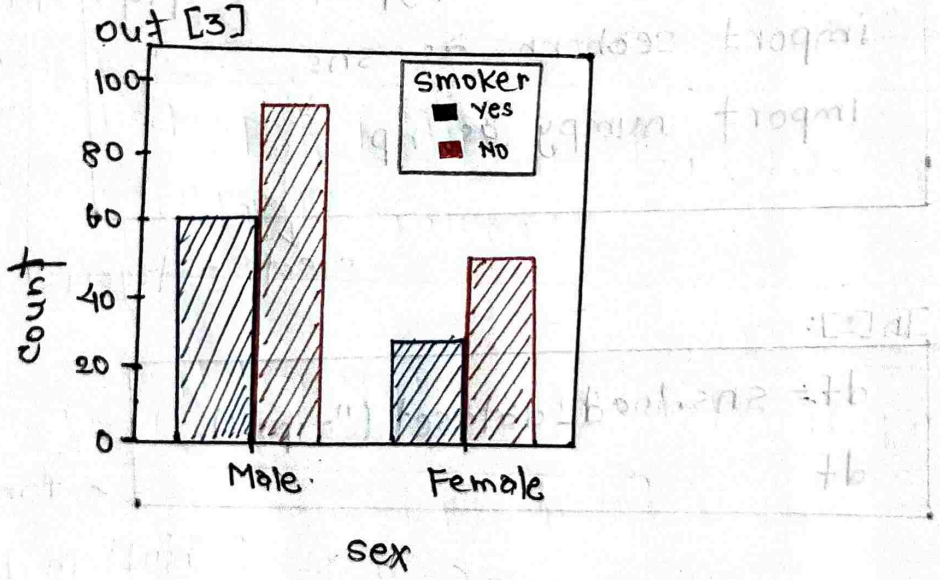
Used to shows the number of occurances (frequency) of each category in a dataset.

• "hue" Parameter: this parameter divides each category into subcategories using another categorical variable.

ex code =

```
In [3]:
sns.countplot(x=df['sex'], hue = df['smoker'])
```

output: <Axes : xlabel = 'sex' , ylabel = 'count' >



→ Bar Plot (sns.barplot):

It compares a categorical variable with a numeric variable.

- X-Axis = categorie
- Y- Axis = Numerical values

• **Default Behaviour (Mean)** : By default, sns.barplot() calculates the mean (Average) of the numerical variable for each category.

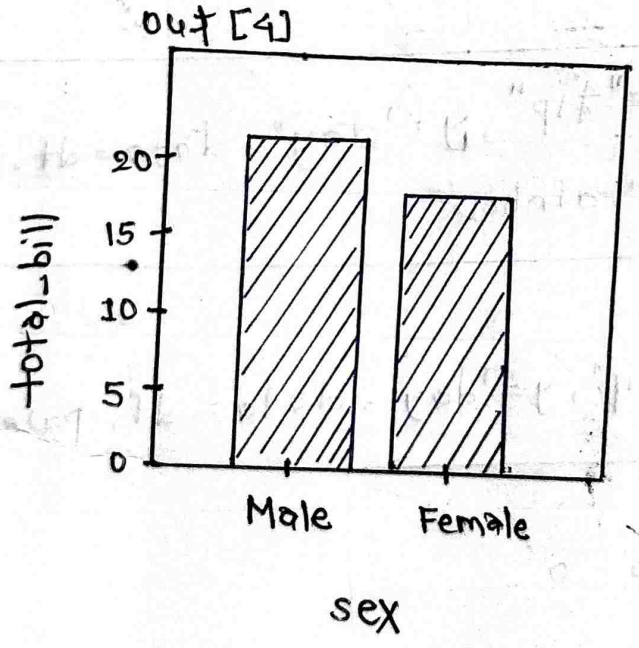
• **The estimator Parameter:** We can change what statistic is displayed using the "estimator" argument.

ex code =

In [4]

```
sns.barplot(x= df['sex'], y = df['total_bill'])
```

Output = <Axes : xlabel ='sex' , ylabel = 'total_bill' >



→ Box Plots & Violin Plots

Both used to show the distribution of Quantitative data across different levels of Categorical variable.

(1) Box Plot (sns.boxplot): shows the distribution of numerical data using quartiles.

- ↓ divide sorted data into 4 equal parts
- The box represents the Middle 50% of data (IQR).
- The line inside the box is the meadian.
- IQR measures the spread of the Middle 50% of the data

ex code =

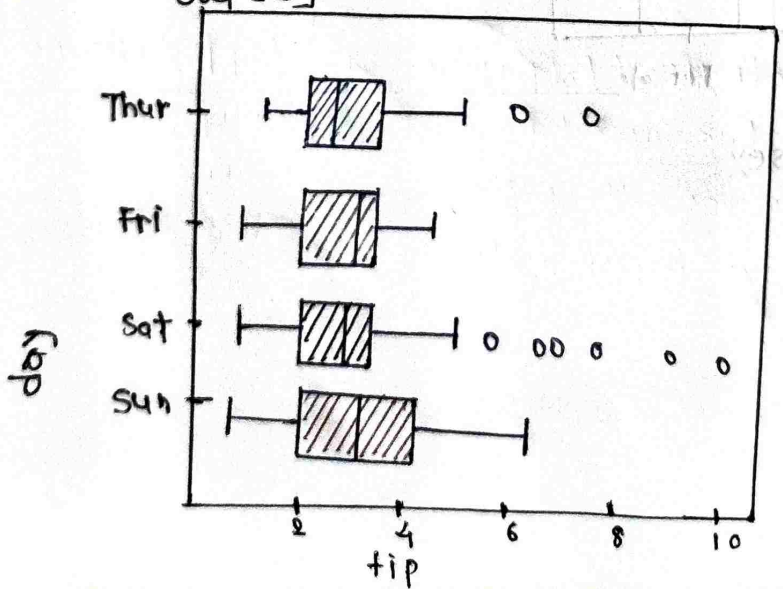
In [5]:

```
sns.boxplot(x="tip", y="day", data=df,
            Palette="rainbow"
```

output:

```
sns.boxplot(x='tip', y='day', data = df, palette='rainbow')
```

out [5]



(2) violin plot (sns.violinplot): It serves a similar purpose to a box plot but provides a different visual representation.

It physically widens in area where the data points are denser & narrows where are fewer data points.

ex code =

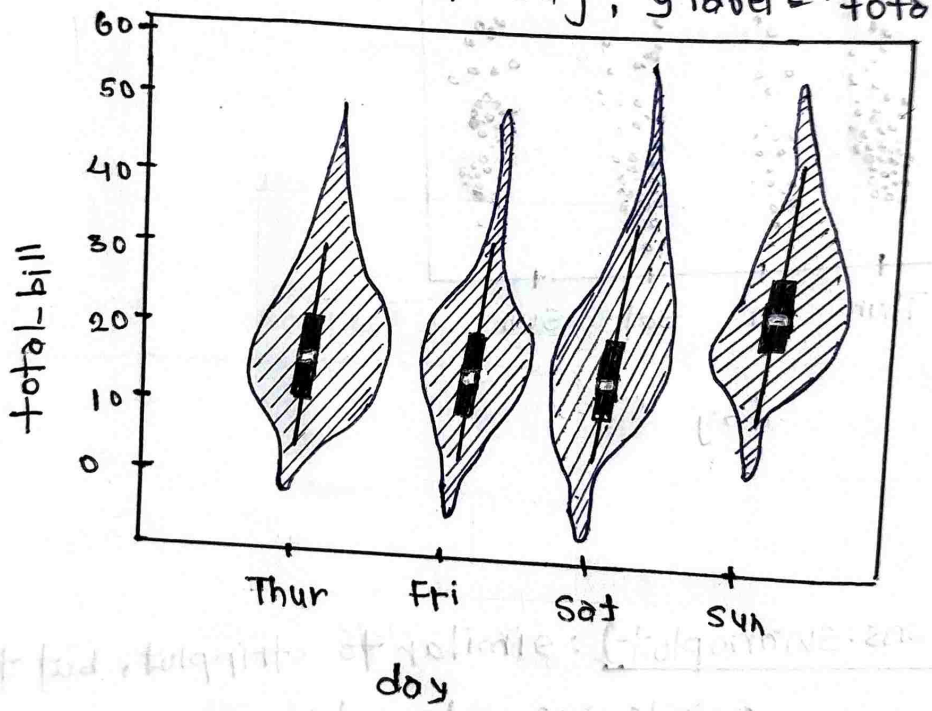
In [6]

```
sns.violinplot(x="day", y="total_bill",
data=df)
```

Output =

Out[6]: only for representation, Not 100% Accurate

<Axes : xlabel='day', ylabel='total_bill'>



→ Strip Plots & Swarm Plots : Used for display individual data points rather than just aggregate shapes They are often used as a good complement to box or voilin plots.

(1) Strip plot (sns.stripplot) : It draws a simple scatter plot where one variable is categorical.

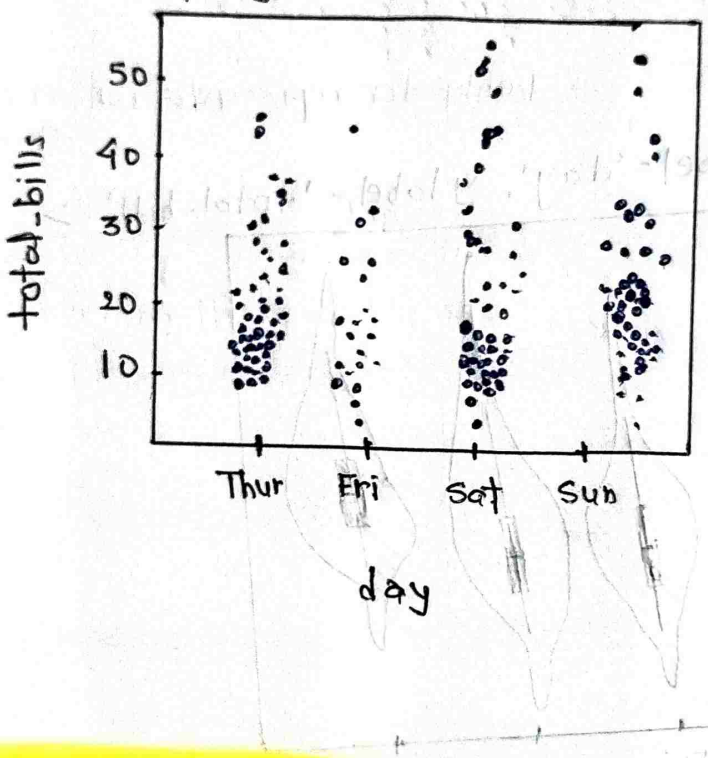
ex code =

In [7]:

```
sns.stripplot(x='day', y='total_bill', data=df)
```

Output : <Axes: xlabel='day', ylabel='total_bill'>

Out [8]:



(2) Swarm Plot (sns.swarmplot) : similar to stripplot, but the points are automatically adjusted along the categorical axis so that they do not overlap.

It gives a much clearner representation of data density.

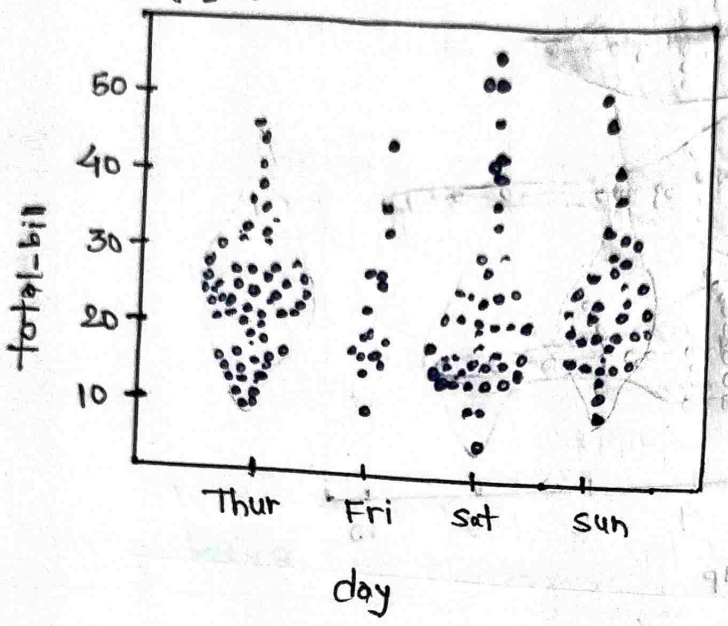
In [9]:

ex code =

```
sns.swarmplot (x='day', y='total_bill', data=df)
```

Output: <Axes: xlabel='day', ylabel='total_bill'>

Out [9]:



→ Its shapes looks like violin plot

→ **Advance Visualization Tip**: We can combine plots by writing the code for violin & swarm plot on the same canvas.

This allow us to see the overall shape of the data distribution while also seeing exactly where every single non-overlapping data point lies.

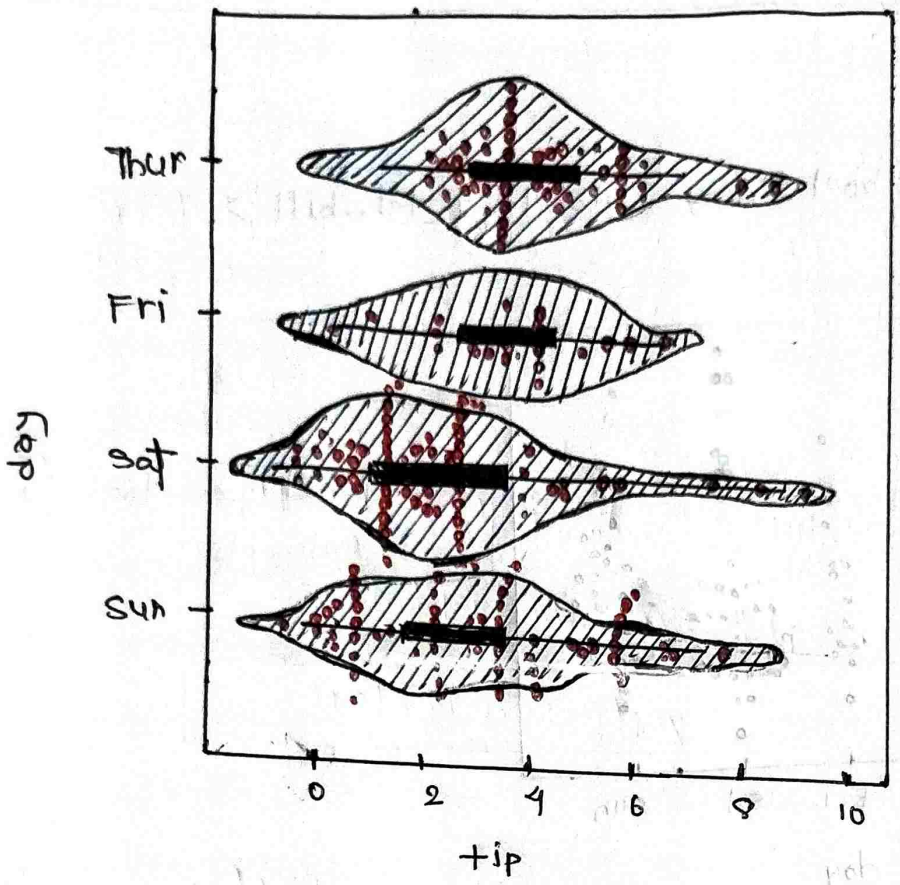
In [10]:

ex code =

```
sns.violinplot (x='tip', y='day', data = df)
sns.swarmplot (x='tip', y='day', data=df)
```

Output:

<Axes : xlabel = 'tip', ylabel = 'day'>



* while running the code it shows warning or UserWarning but that's fine. code run

4. MATRIX PLOTS

→ Core Concept

Traditional plots that use standard x & y axes to plot individual data points, a matrix plot relies on a 2D matrix structure.

- **Working:** It uses a grid of rows & columns where the intersecting cells contain data values represented by different colors.

→ Preparing Data For Matrix Plots

To create a matrix plot in seaborn, our data must already be in a matrix format.

first we load a built in dataset

In[1]:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

In[2]:

```
flights = sns.load_dataset('flights')
flights
```

In[3]:

```
tips = sns.load_dataset("tips")
tips
```

(1.) Correlation Matrix (.corr()) : Correlation measures how strongly two variables related to one another.

for finding the Correlatim, isolate the numerical column in our dataset & apply .corr() method.

In [4]:

ex code =

```
#Extracting numerical column & calculating correlation
tipscorr = tips [['total_bill', 'tip', 'size']]
correlation_matrix = tipscorr.corr()
correlation_matrix
```

output = out[4]:

	total_bill	tip	size
total_bill	1.000000	0.675734	0.598315
tip	0.675734	1.000000	0.489299
size	0.598315	0.489299	1.000000

(2.) Pivot tables (.pivot.table()): It allow us to manually decide what goes on the rows (index), what goes on the columns, & what value fill the grid.

In [5]

ex code =

```
Pvt flight = flight.pivot_table(values='passenger',
index = 'month', columns = 'year')
Pvt flight
```

→ Heat Map (sns.heatmap)

A graphical representation of data where values are shown using different colors.

- Darker / warmer colors → higher values
- Lighter / cooler colors → lower values

It translate our matrix data into a color-coded grid.

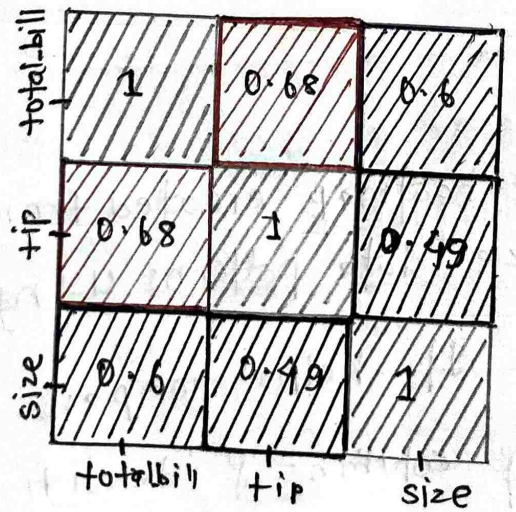
"annot" Parameter: By default, we only see colors. If we add annot=True, seaborn will overlay the exact numerical correlation values inside the colored boxes.

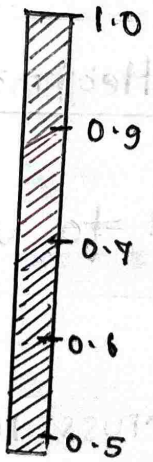
ex code =

In [6]:
```
sns.heatmap(tipscorr.corr(), annot = True)
```

Output =

Out [6]:





→ Cluster Map :-

It is a heat map that automatically group similiar rows & column using clustring.

It helps to find patterns & relationship in data.

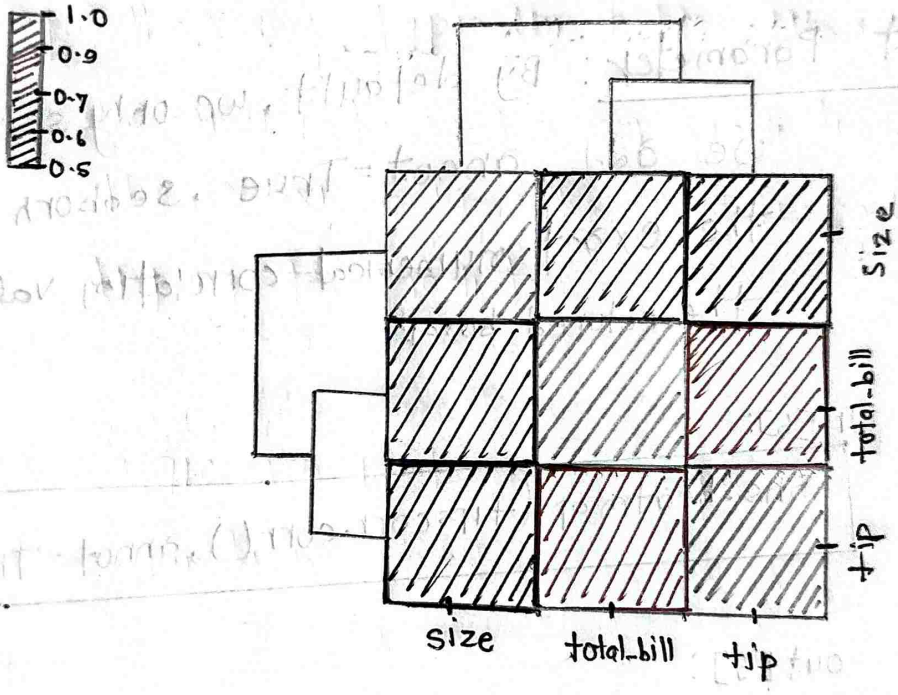
ex code =

In [7]:

```
sns.clustermap(tipscorr.corr())
```

Out [7]:

output =



→ Pivot table Heatmap

A heatmap created from a pivot table to visualize data patterns using colors.

It helps compare value across rows & columns & identifies trends easily.

Ex code =

In [8]:

```
pvtflight = flight.pivot_table(values='passengers',
index='month', columns='year')
pvtflight
```

In [9]

```
sns.heatmap(pvtflight)
```

output = <Axes : xlabel='year', ylabel='month'>

can't Make heatmap here -

Dark color = low values

light color = high values

The heatmap shows passenger growth over the years, with peak values in July - August.

5. REGRESSION PLOTS

→ **Core concept**

regression plot is used in Machine learning so we will not fully discuss what, how & where it is used but we will see how it is created & what are values we can put inside it

- **Axis Mapping** - the x-Axis typically holds the data we are using to make our prediction, while the y axis holds the specific value we trying to predict.

* To create a regression plot in seaborn, we use the "lmplot()" function.

lets load -

In [1]:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

In [2]:

```
tips = sns.load_dataset("tips")
```

→ Regression plot:

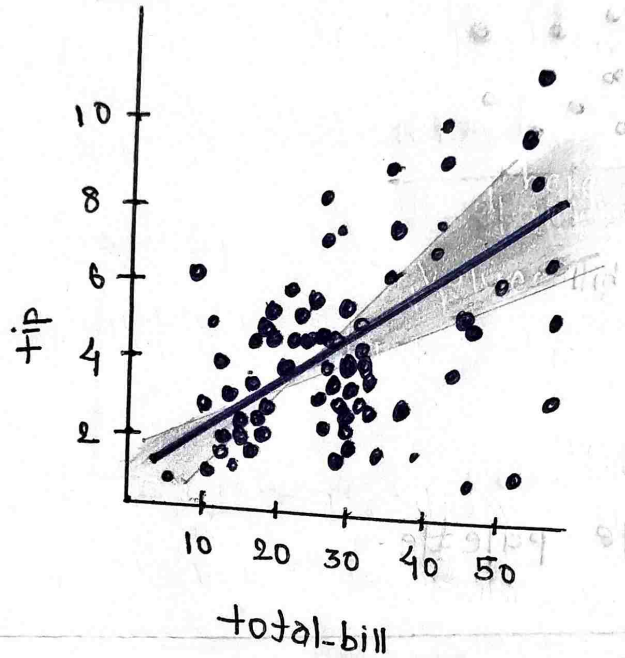
In [3]:

ex code =

```
sns.lmplot(x='total_bill', y='tip', data=tips)
```

Out[3]:

output = <seaborn.axisgrid.FacetGrid at 0x12c0ac650>



• **Using 'hue' Parameter** : we can use hue parameter to add a categorical variable to the plot & creates seperate regression lines for each category.

In [4]:

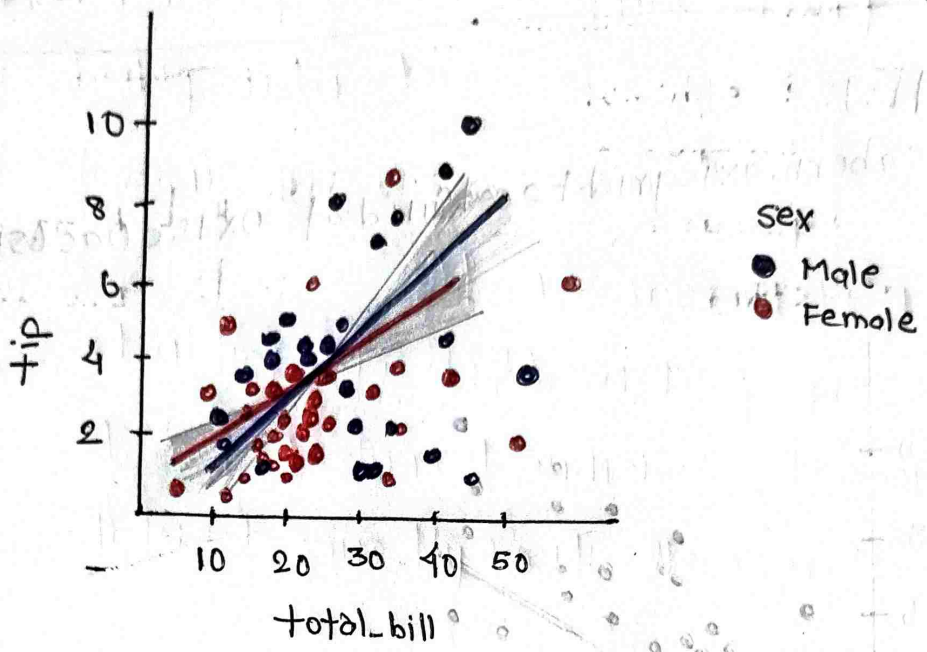
ex code =

```
sns.lmplot(x='total_bill', y='tip', data=tips,
hue='sex')
```

Output =

Out[4]:
<seaborn.axisgrid.FacetGrid at 0x30f28f2f0>

only for representation, Not 100% Accurate



* We can also change palette.

ex code =

In [5]:

```
sns.lmplot(x='total_bill', y='tip', data=tips,
hue='sex', palette='coolwarm')
```

→ Customizing Markers

Just like standard scatter plots, we can customize the shapes representing our data points using the "Markers" parameter.

ex code =

In [6]:

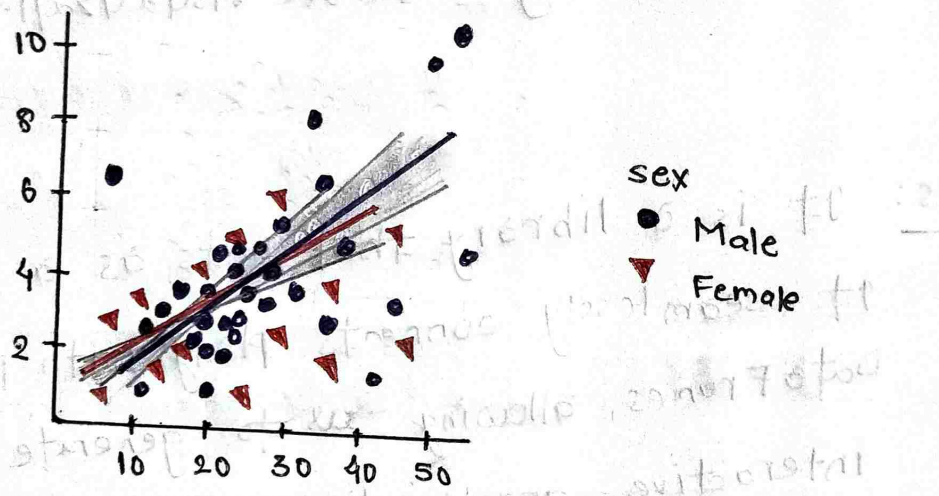
```
sns.lmplot(x='total_bill', y='tip', data=tip,
hue='sex', palette='coolwarm',
markers=['o','v'], scatter_kws={'s':50})
```

→ Uses circle (o) for one category & triangle (v) for the other

→ Sets scatter point size to 50 (s=size)

output =

```
<seaborn.axisgrid.FacetGrid at 0x30fb1c0e0>
```



6. PLOTY & CUFFLINKS

→ **Plotly**: It is open-source Python library used to design highly interactive, clickable & zoomable charts. (even in 3D).

It is incridibly useful for building data dashboards & presenting data to non-technical audiences because we can interact directly with the visualization.

→ **cufflinks**: It is a library that acts as a bridge. It seamlessly connects ploty with pandas DataFrames, allowing us to generate plotly's interactive graphs directly from our pandas data with just one line of code.

→ **Installation & setup**

In[1]

```
!Pip install Plotly
!Pip install cufflinks
```

- #for installing

In [2]:

```
import seaborn as sns
import pandas as pd
import cufflinks as cf
from plotly.offline import iplot
```

→ The ".iplot()"

To create interactive charts, we replace standard plotting commands with ".iplot()" method directly on our Pandas DataFrame or series.

- (A) Interactive Line graph :

first we load 'tips'

In [3]:

```
tips = sns.load_dataset('tips')
tips
```

Now

By simply calling ".iplot()" on numerical column, it generates an interactive line graph where we can hover over any point to see its exact value, zoom in on specific areas, and auto-scale.

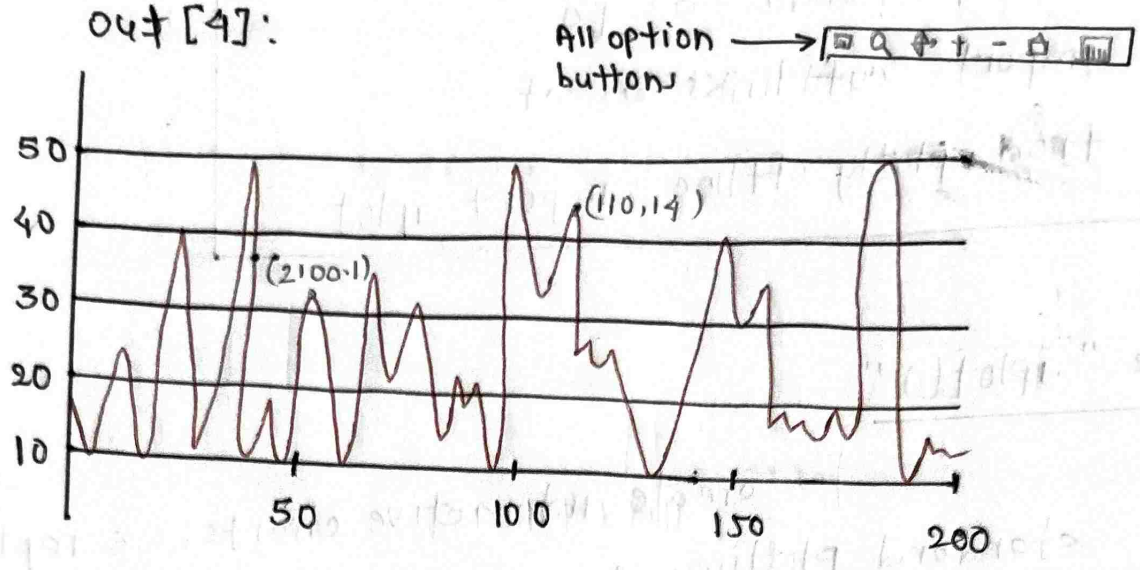
In [4]:

ex code =

```
tips['total_bill'].iplot()
```

output: only for representation, Not 100% Accurate

Out [4]:



(B) Interactive Bar Plot

To create a meaningful bar plot, we usually group our data first.

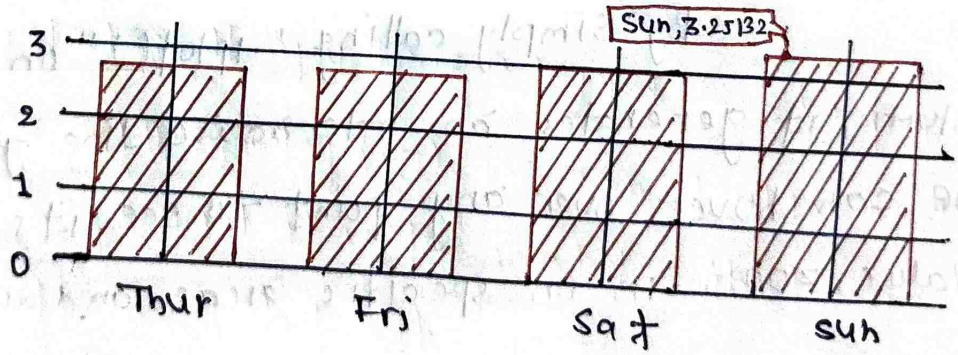
ex code >

In [5]:

```
tips.groupby('day')['tip'].mean().iplot(kind='bar')
```

output :

Out [5]:



•(C) Interactive Scatter Plot

specify x, y & kind='scatter', & use mode='markers' to display points.
without mode='marker', Ploty connects the points with lines.

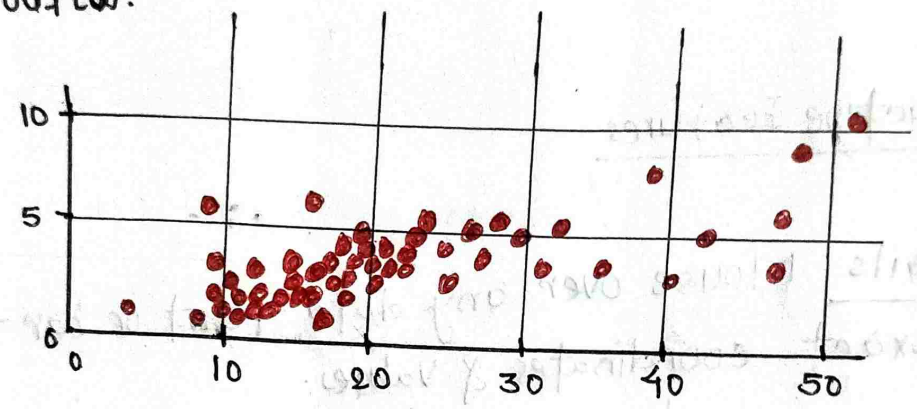
ex code =

In [6]:

```
tips.iplot(kind='scatter', x='total_bill', y='tip',
 mode='markers')
```

Out [6]:

output:



* Not 100% Accurate.

•(D) Interactive Box plot

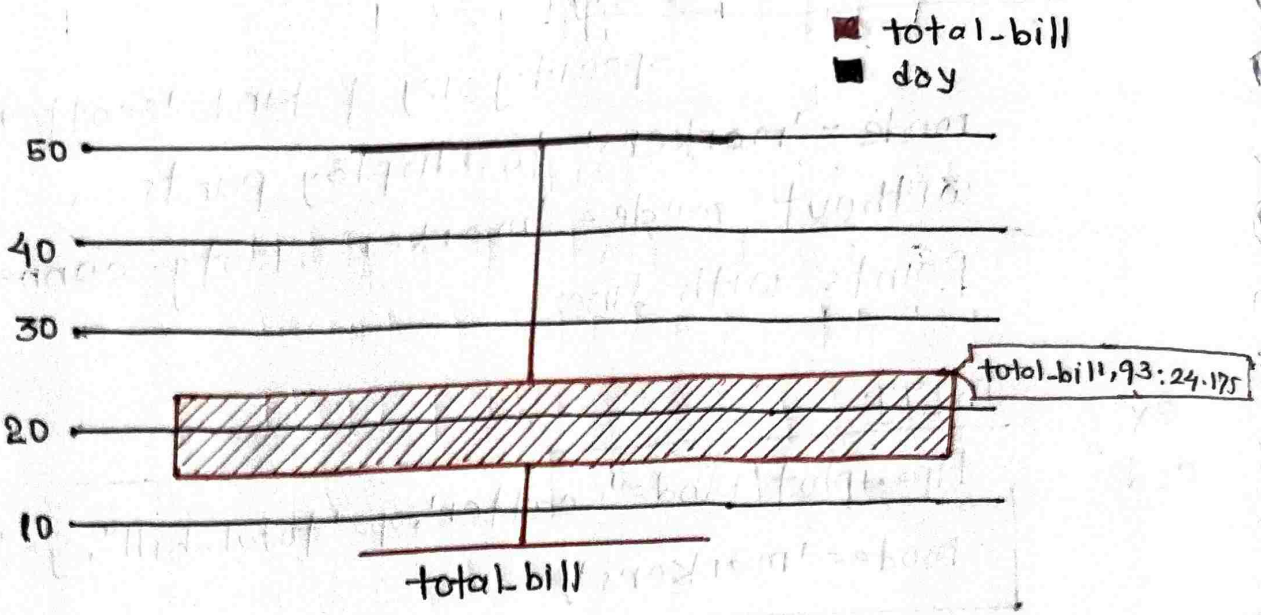
use "kind='box'" to visualize data distribution. Hovering over a box displays key statistics such as minimum, Q1, median(Q2), Q3, & Maximum values.

In [7]:

ex code =

```
tips[['total_bill', 'day']].iplot(kind='box',
 x='total_bill', y='day')
```

Output =



→ Key Interactive Features

- **Hover Details**: Mouse over any data point or bar to see exact coordinates & values.

- **Zooming / Panning**: click & drag a box over a section of chart to zoom in, & use the auto-scale button to reset the view.

- **Isolating Data**: If you have a legend (ex- comparing multiple categories), we can double-click on a legend item to isolate that specific track & hide the rest.

• **Export:** We can easily export the generated interactive plot as a static PNG image right from the chart's tool menu.

IPL 2022 CAPSTONE PROJECT

This project explores IPL 2022 Match-level data to derive meaningful insights & Understand match outcomes, player performances, & team dynamics.
The data is in "IPL.csv".

SOLUTION:-

→ (1) Data Loading & Exploration

- We import

```
import Numpy as np
import Pandas as pd
import Matplotlib.pyplot as plt
import seaborn as sns
```

- We load the IPL dataset

```
Pd.read_csv()
```

- We use `df.info()` to check data types.
- We use `df.shape` to find dataset dimensions (74 row, 20 columns)

- We use `df.isnull().sum()` to check missing values & find none.

→ Team with Most wins

- We apply `value_counts()` on `match_winner`
- We visualize the result using `sns.barplot()`
- Gujarat Titans won the most matches (12).

→ Toss Decision Trend

- We use `sns.countplot()` on `toss_decision`
- Most teams chose to field after winning the toss.

→ Toss Impact

- We compare `toss_winner` & `match_winner`
- We calculate the winning percentage.
- Toss-winning teams won only 48.65 % of matches.

→ Runs Vs Wickets

- We use `countplot()` on `won_by`
- 37 matches were won by runs & 37 by wickets.

→ Player of the Match

- We use `value_counts().head(10)`
- Kuldeep Yadav received the most awards (4).

→ Top scorers

- We group data by `top_scorer`
- we sum high `high_score` & sort in descending order
- Jos Buttler & Quinton de Kock were the top scorers.

→ Best Bowler

- We extract wickets from best_bowling using `split('--')`
- We convert values to integer & aggregate wickets.
- Yuzi chahal took the most wickets (17).

→ Venue Analysis

- We apply `value_counts()` on venue.
- Wankhede stadium hosted the most matches (20).

→ Additional Insights

- Highest victory margin : chennai by 91 runs.
- Highest individual score : Quinton de kock (140).